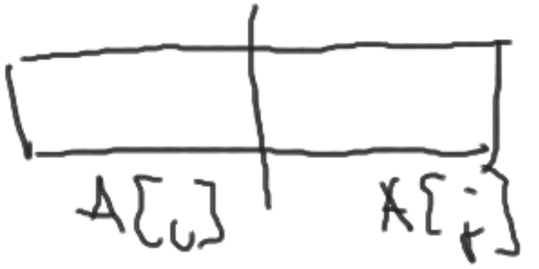
Maximális növekedés



A[1:n] -> keressünk olyan 1<=i<=j<=n indexeket, amelyekre A[j]-A[i] maximális!

Oszd meg és uralkodj megközelítés

1) Bontsuk a feladatot két "feleakkora" méretű feladatra -> A[1:q] és A[q+1:n]

2) Határozzuk meg rekurzívan a maximális növekedést a résztömbben:

A[1:q] -> 1<=i'<=j'<=q

A[q+1:n] -> q+1<=i"<=j"<=n

Egyelemű tömb esetén az egy elem indexét adjuk vissza mindkét visszatérési értékként

3) A maximális növekedés lehet

I. A[1:q]-ban van -> i=i' és j=j'

II. A[q+1:n]-ben van -> i=i" és j=j"

III.

1<=i<=q és q+1<=j<=n

A[i] A[1:q] minimuma

A[j] A[q+1:n] maximuma

Válasszuk a legkedvezőbbet!

Költség:

$$T(n) = 2T(n/2) + \Theta(n)$$

két rekurzív hívás

min kiv. A[1:q] és maxkiv. A[q+1:n]

$$T(1) = \Theta(1)$$

Ez ugyanaz a rekurzió, mint az összefésüléses rendezésnél

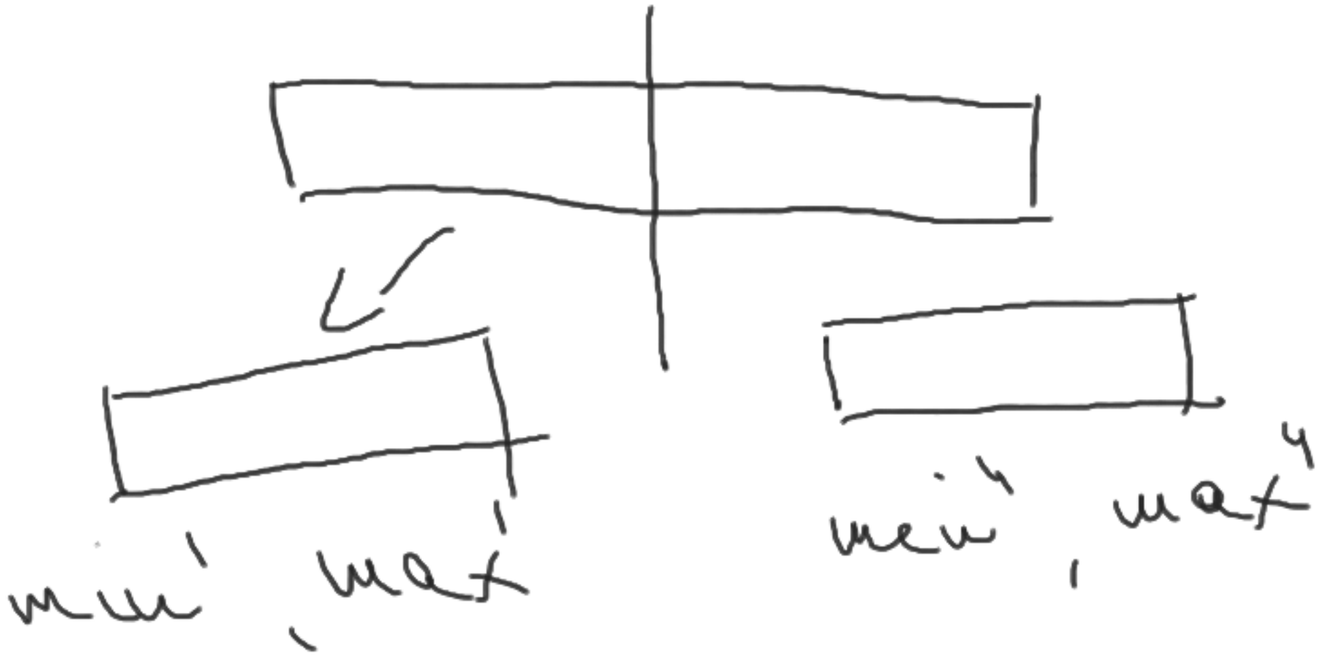
$$\Rightarrow \quad T(n) = \Theta(n \log n)$$

$$\Downarrow$$

$$T(n) = 2T(n/2) + \Theta(1)$$

$$\Theta(n)$$

Ez elérhető úgy, hogy a min és a max elemeket is rekurzívan számoljuk:



max ← max', max''

min ← min', min''

konstans költséggel

Egyelemű tömb esetén mind a 4 visszatérési érték ugyanaz az index.

```
JavítottMaximálisNövekedés(A,l,r)
    if l=r then return (l, l, l, l)
    q=[(l+r)/2]
    (x, x', l_min, l_max)=JavítottMaximálisNövekedés(A,l,q)
    (y, y', r_min, r_max)=JavítottMaximálisNövekedés(A,q+1,r)
    z=l_min
    z'=r_max
    if A[x']-A[x]>=A[y']-A[y] AND A[x'] - A[x] >= A[z']-A[z]
            then return (x, x', min(l_min, r_min), max( l_max, r_max))
    if A[y']-A[y]>= A[x']-A[x] AND A[y'] - A[y] >= A[z'] -A[z]
            then return (y, y', min(l_min, r_min), max(l_max, r_max))
    if A[z']-A[z]>=A[x']-A[x]  AND A[z']-A[z]>= A[y'] -A[y]
            then return (z, z', min(l_min, r_min), max(l_max, r_max))
```

Többségi elem

A[1:n]-ben egy elemet többségi elemnek nevezünk, ha több, mint n/2-ször fordul elő (max egy többségi elem lehet!)
Adjunk hatkony algoritmust annak eldöntésére, hoy van-e többségi elem A[1:n]-ben

Oszd meg és uralkodj megközelítés
(nem lehet <,> összehasonlítást végezni)

1) Bontsuk a feladatot két feleakkora részfeladatra: A[1:q] és A[q+1:n]
2) Rekurzívan döntsük el, hogy van-e többségi elem A[1:q]-ban és A[q+1:n]-ben

A[1:q] -> x' (x'="NIL" előfordulhat)
A[q+1:n] -> x" (x"="NIL" előfordulhat)

3) Vegyük észre, hogy ha x többségi elem A[1:n]-ben, akkor többségi elem A[1:q] és A[q+1:n] legalább egyikében, így x=x' vagy x=x"
Számoljuk össze x' és x" előfordulásait A[1:n]-ben és ha valamelyik n/2-nél többször van jelen, akkor nyilvánítsuk őt többségi elemnek.

Költség:

$$T(n) = \begin{cases} 2T(n/2) + O(n) & \leftarrow \text{„ök"} \\ O(1) & \text{ha } n=1 \end{cases}$$

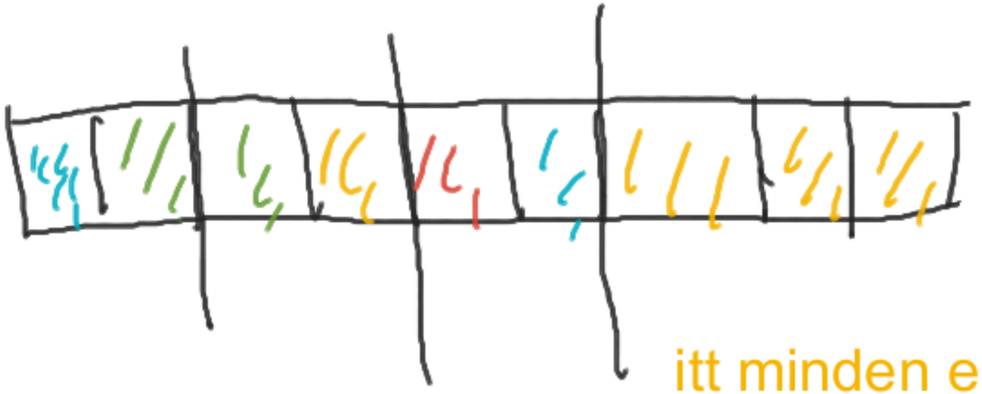
$$\Rightarrow T(n) = O(n \log n)$$

Hogyan lesz ebből lineáris költség?

Észrevétel:

Ha x többségi elem A[1:n]-ben és A[1]≠A[2] -> x többségi elem A[3:n]-ben is

Rendezzük át ezek után az A[1:n]-et így:



ezekben a párokban az elemek különbözők

itt minden elem ugyanaz -> csak ez lehet többségi elem ELLENŐRZÉS!

Hogyan lehet ezt az átrendezést gyorsan megcsinálni?

Ism: gyorsrendezés:

Ha [illegible] > [illegible] → helyben marad

Ha [illegible] < [illegible] → felcseréljük az első [illegible] -del

A többségi elemnél hasonló, amit csinálunk:

itt tartunk

homogén rész

Ha [illegible] = [illegible] → marad a helyén

Ha [illegible] ≠ [illegible] → felcseréljük a 2. [illegible] -ml

```
TöbbségiElem(A)
    i=0
    j=1
    while j<=n do
          if A[j]<>A[i+1] then
                i=i+2
                Csere(A[j],A[i])
          j=j+1
    if i=n
       then return nil
    else
        s=0
        for k= 1 to n do
              if A[k]=A[i+1] then s=s+1
         if s>n/2
              then return A[i+1]
              else return nil
```

A[1:n]-ben pozitív és negatív egészek, sőt 0 is. Határozzuk meg azokat az 1<=i<=j<=n indexeket, melyekre A[i]+A[i+1]+ .... + A[j] maximális

$\rightarrow O(n \log n)$

$\rightarrow O(n)$